



Green MLOps:
Framework for Energy-Efficient Machine Learning Operations
Pipelines

Case Study Report

Nandan Muruli (ID: 100004359)
Master's in Applied Computer Science
SRH University Heidelberg

Supervisor: Prof. Dr. Kamellia Reshadi

September 2026

Table of Contents

Abstract	3
1. Introduction & Problem Statement	3
2. Related Work	4
3. Research Questions	6
4. Methodology & Architecture	6
5. Implementation	7
6. Experimental Results	10
7. Discussion	13
8. Limitations	14
9. Conclusion & Future Work	15
References	16
Bibliography	17
Appendix A: Interpreting the MLflow Experiment-Tracking Table	17
Appendix B: Full Pipeline Configuration (configs/config.yaml)	20
Appendix C: Raw Result Tables (Full Precision)	21
Appendix D: Reproducing This Study End-to-End	21
Appendix E: Glossary of Terms and Abbreviations	22

Abstract

Deep learning's growing compute demand carries a carbon cost that conventional MLOps pipelines do not track. This case study designs, implements, and evaluates a “Green MLOps” pipeline that treats energy consumption as a first-class metric alongside model accuracy, using a DistilBERT sentiment-classification task on the IMDB dataset as the workload. The pipeline integrates DVC for data/pipeline versioning, MLflow for experiment tracking, CodeCarbon for energy/emissions telemetry, and Docker/GitHub Actions for containerized, CI-driven execution. Three research questions are investigated: (RQ1) how power-consumption tracking can be integrated non-intrusively into automated training loops; (RQ2) the quantitative accuracy-energy trade-off under model compression (dynamic quantization, magnitude pruning); and (RQ3) whether data-driven scheduling and execution-level resource capping reduce emissions without violating latency service-level agreements (SLAs). Because the available hardware has no Intel RAPL interface and no NVIDIA GPU, the originally proposed Scaphandre telemetry and GPU power-capping components were substituted with CodeCarbon-only tracking and a CPU-thread-capping proxy respectively — a documented, hardware-driven scope adaptation. Results show INT8 dynamic quantization cuts inference latency by 53% and model size by 48% for a 6.3-percentage-point accuracy cost; unstructured pruning at 30% sparsity yields no latency or size benefit without a sparse-compute runtime; execution-level thread-capping trades latency for lower measured energy draw while remaining within a 500ms SLA; and an early-stopped training run reached a substantially better validation loss than brute-force grid search while consuming 23.5% less energy.

1. Introduction & Problem Statement

The exponential growth of deep learning has led to a substantial increase in the carbon footprint of data centers. Traditional Machine Learning Operations (MLOps) pipelines optimize heavily for model accuracy, inference latency, and throughput, while largely ignoring environmental sustainability. There is a gap in software engineering frameworks that seamlessly integrate energy-consumption metrics into automated CI/CD pipelines for machine learning.

This project addresses that gap by designing, implementing, and evaluating a Green MLOps pipeline that treats energy efficiency as a first-class citizen alongside model accuracy, using an end-to-end sentiment-classification case study (DistilBERT fine-tuned on the IMDB movie-review dataset) as the concrete workload against which every measurement in this report was taken.

2. Related Work

Three overlapping bodies of work motivate and inform this project's design: methods for measuring and reporting the energy/carbon cost of ML workloads, the broader Green AI research agenda that treats efficiency as a first-class objective rather than an afterthought, and the model-compression techniques this project applies to a concrete inference workload in RQ2. This section situates the pipeline described in §4-5 against each, and closes by positioning this project's specific contribution relative to the closest prior work.

2.1 Measuring and reporting ML energy costs

Strubell, Ganesh & McCallum (2019) first drew wide attention to the compute and carbon cost of NLP research by estimating the cloud-compute and CO₂e cost of training several large Transformer models, work that catalyzed later measurement tooling rather than proposing a tool itself. Henderson et al. (2020) responded with a standardized “Experiment Impact Tracker” and reporting checklist, arguing that energy/carbon figures are only comparable across studies if the measurement methodology -- sampling interval, hardware inventory, regional grid carbon intensity -- is disclosed alongside the number itself. That concern is directly reflected in this report's §8 discussion of CodeCarbon's 1-second sampling resolution and its effect on sub-5-second evaluation runs: without disclosing that resolution, the RQ2/RQ3 emissions figures in §6 would be uninterpretable rather than merely noisy. Lacoste, Luccioni, Schmidt & Dandres (2019) introduced the mlco₂ “Machine Learning Emissions Calculator” and the underlying measurement approach -- live hardware power draw combined with a region-specific grid carbon-intensity factor -- that CodeCarbon (the tool used throughout this project) directly descends from; that lineage is visible in this project's own logs/emissions.csv, whose country_name/country_iso_code/region columns are exactly the grid-intensity inputs their method requires. Anthony, Kanding & Selvan (2020) released Carbontracker, a comparable tool that additionally predicts a training run's total footprint from its first few epochs; CodeCarbon was used instead in this project mainly for its lightweight Python context-manager API (wrapped here as GreenTracker, §5.2) and because its core CPU energy estimate does not require Intel RAPL access, which §4.2 documents this project's hardware as lacking.

2.2 Efficiency, scale, and Green AI

Schwartz, Dodge, Smith & Etzioni (2020) coined the term “Green AI” to distinguish research that treats efficiency as a first-class evaluation criterion from “Red AI”, which chases state-of-the-art results primarily through ever-larger compute budgets, and proposed reporting FLOPs and energy alongside accuracy as a community norm -- the same principle this project's MLflow schema operationalizes at the level of individual pipeline runs rather than only in a paper's results table

(Appendix A). Patterson et al. (2021) quantified the carbon footprint of training several large language models and showed that model architecture, datacenter, and energy-grid choices can shift total emissions by orders of magnitude, establishing that engineering and operational choices are a legitimate efficiency lever in their own right, not merely a footnote to algorithmic ones -- a framing this project's RQ3 adopts directly by studying scheduling and execution-level knobs rather than only model-level ones. Desislavov, Martínez-Plumed & Hernández-Orallo (2023) studied inference-time (rather than training-time) energy trends specifically, and found that inference cost scales more gently with model capability than training cost does, with batch size and hardware utilization dominating per-query energy more than raw parameter count -- directly relevant to why this project measures inference latency and energy under thread caps and a fixed batch size (RQ3a, §6.2) rather than only training-time telemetry (RQ1, §5.2). Verdecchia, Sallou & Cruz (2023) conducted a systematic review of 98 Green AI studies and found that most measure a single model or algorithm in isolation, with comparatively few embedding energy measurement inside a reproducible, versioned MLOps pipeline -- a gap this project's overall design (DVC + MLflow + CodeCarbon, §5) is built to close.

2.3 Model compression for efficient inference

Han, Mao & Dally (2016) combined pruning, trained quantization, and Huffman coding sequentially in their “Deep Compression” pipeline to shrink CNN model size by an order of magnitude with negligible accuracy loss, establishing pruning and quantization as complementary rather than competing techniques -- provided the deployment runtime can actually exploit the resulting sparsity. That proviso matters for interpreting this project's own RQ2 result (§6.1): quantization produced a substantial latency and size win here, but unstructured pruning alone did not, which is consistent with Han et al.'s pipeline rather than a contradiction of it, since their pruning gains depend on a sparse-aware storage/compute path that this project's evaluation setup -- dense PyTorch tensors, no sparse kernel -- does not use. Jacob et al. (2018) formalized integer-only quantized inference, the general approach that PyTorch's `torch.quantization.quantize_dynamic` (used in `compress.py`, §6.1) implements a dynamic variant of, and reported 2–4x throughput gains on general-purpose CPU and mobile hardware without any dedicated accelerator -- consistent with this project's own measured 53.5% latency reduction from INT8 dynamic quantization on CPU-only hardware.

2.4 Positioning this project

Closest in spirit to this project is Sánchez-Mompó et al. (2025), “Green MLOps to Green GenOps,” an empirical study measuring energy consumption across discriminative and generative model operations using software-based power measurement on real hardware; they find that

architecture, hyperparameter, and hardware choices can materially reduce energy without hurting performance for discriminative models, and that large-language-model efficiency depends more on utilization than on raw parameter count. This project shares that empirical, tooling-based methodology but differs in the depth of pipeline integration: rather than benchmarking models as standalone scripts, this project embeds the same class of telemetry (CodeCarbon) inside a versioned, resumable, CI-tested MLOps pipeline -- DVC for data/stage versioning, MLflow for per-run tracking, GitHub Actions for continuous smoke-testing (§5) -- so that every energy measurement is attached to a specific, reproducible pipeline run rather than a one-off benchmarking invocation. That distinction directly addresses the tooling-integration gap Verdecchia et al. (2023) identify in the wider Green AI literature, and it does so on hardware -- Apple Silicon for development, a CPU-only Linux environment for execution, with no Intel RAPL interface and no NVIDIA GPU -- that none of the studies cited in this section specifically target, which is itself the documented scope adaptation discussed in §4.2.

3. Research Questions

RQ1: How can power-consumption tracking tools be non-intrusively integrated into automated ML training and deployment loops?

RQ2: What is the quantitative trade-off between model performance (F1-score) and energy consumption when applying model compression techniques?

RQ3: Can data-driven scheduling and execution-level resource capping achieve a meaningful carbon reduction without violating operational SLA latency?

4. Methodology & Architecture

The project follows an empirical case-study approach, building a fully functional, locally reproducible pipeline rather than a simulated one. The workload baseline is a DistilBERT (distilbert-base-uncased) binary sentiment classifier fine-tuned on a subset of the IMDB dataset (500 training / 100 evaluation examples, 2 epochs), matching the scale needed to keep every stage reproducible within the project's time budget while still exercising a realistic transformer fine-tuning workload.

4.1 Tech stack

Layer	Tool	Role in this project
Pipeline & versioning	DVC	Tracks the raw dataset and defines the ingest → train → compress → RQ3 → dashboard stage graph (dvc.yaml / dvc.lock)

Layer	Tool	Role in this project
Experiment tracking	MLflow	Logs parameters, accuracy/F1/latency metrics, and carbon-footprint metrics per run (sqlite:///mlflow.db)
Energy telemetry	CodeCarbon	Process-level power/energy/emissions estimation, wrapped in a reusable GreenTracker context manager
Containerization	Docker	Packages the pipeline for CI or standalone containerized execution
CI/CD	GitHub Actions	Lints and smoke-tests the pipeline code on every push/PR
Dashboard	MLflow UI + static HTML	Live run inspection plus a summary dashboard for the three research questions

4.2 Documented scope adaptation

The original expose specified Scaphandre (an Intel-RAPL-based bare-metal/Kubernetes power-telemetry tool) and GPU power-capping for RQ3. The development and execution hardware available for this project — an Apple Silicon machine for development, and a CPU-only Linux environment used to run the experiments reported here — has no Intel RAPL interface and no NVIDIA GPU, so neither tool is applicable. This is a hardware constraint, not a design choice, and is treated as an explicit, stated limitation of the study rather than left implicit:

- Energy telemetry uses CodeCarbon only (process/CPU/RAM-level estimation), not Scaphandre.
- “Execution-level power capping” (RQ3) is implemented as a CPU-thread-count cap (`torch.set_num_threads`), a software-controllable proxy for the GPU power-capping originally proposed, benchmarked against a defined SLA latency threshold.
- The dashboard combines MLflow's built-in UI with a static HTML summary rather than a full Grafana/Kubernetes deployment.

5. Implementation

5.1 Pipeline & versioning

The raw IMDb dataset is tracked via ``dvc add`` (`data/raw/imdb.dvc`); the processing pipeline (`ingest` → `train_baseline` → `compress` → `rq3_experiments` → `dashboard`) is declared in `dvc.yaml` so that ``dvc repro`` reproduces every stage from a clean checkout, and ``dvc dag`` renders the dependency graph. All run parameters (dataset sizes, model name, training hyperparameters, compression settings, SLA threshold, thread caps, grid-search space) live in a single `configs/config.yaml` rather than being hardcoded per script.

The stage graph itself (`dvc.yaml`) makes the `ingest` → `train_baseline` → `compress` → `rq3_experiments` → `dashboard` dependency chain explicit and machine-checkable: each stage

declares its own deps (source files and upstream outputs) and outs/metrics, so `dvc repro` only re-runs a stage -- and everything downstream of it -- when one of its declared dependencies has actually changed, rather than always re-running the full pipeline from scratch. The `train_baseline` stage is a useful illustration of this: it lists `configs/config.yaml` as a dependency precisely so that changing a single hyperparameter (say, `num_train_epochs`) automatically invalidates and re-triggers training, compression, and the RQ3 benchmarks -- without anyone having to remember which downstream stages a given config change affects.

```
stages:
  train_baseline:
    cmd: python -m src.pipeline.train_model
    deps:
      - data/raw/imdb
      - src/pipeline/train_model.py
      - src/telemetry/energy_tracker.py
      - configs/config.yaml
    outs:
      - models/baseline
    metrics:
      - logs/emissions.csv:
        cache: false

  compress:
    cmd: python -m src.pipeline.compress
    deps:
      - models/baseline
      - src/pipeline/compress.py
      - src/pipeline/evaluate.py
    outs:
      - models/quantized
      - models/pruned
    metrics:
      - results/rq2_compression_results.csv:
        cache: false
```

Excerpt of `dvc.yaml` -- two of the pipeline's five stages, showing how config/code/data are declared as dependencies so that `dvc repro` reruns only what actually needs rerunning. (full file reproduced in Appendix B).

5.2 Energy telemetry (RQ1)

RQ1 asks how power tracking can be integrated non-intrusively. The answer implemented here is a GreenTracker Python context manager (`src/telemetry/energy_tracker.py`) that wraps CodeCarbon's EmissionsTracker and automatically logs the resulting carbon-footprint metric to the active MLflow run. Integrating telemetry into any training or evaluation call therefore requires a single `with GreenTracker(project_name=...):` statement around the existing code — no change to the training loop, model code, or MLflow logging calls themselves. Two real baseline training runs captured by this wrapper illustrate the integration:

Run date	Duration (s)	Emissions (mg CO2e)	Energy consumed (Wh)
2026-08-06	41.18	60.96	0.134
2026-08-27	41.47	83.23	0.135

Source: `logs/emissions.csv` (CodeCarbon output), cross-referenced with the corresponding MLflow runs.

Both the standard ML metrics (loss, accuracy) and the carbon-footprint metric appear side-by-side in the same MLflow run, directly answering RQ1: telemetry integration is non-intrusive because it is orthogonal to the model/training code and unifies with the MLOps tracking layer already in place.

The full implementation of that wrapper is short enough to reproduce in full -- which is itself part of the RQ1 answer: non-intrusive integration is credible only if the integration code is small and auditable, not merely described as such.

```
class GreenTracker:
    def __init__(self, project_name="green_mlops_baseline"):
        os.makedirs("./logs", exist_ok=True)
        self.tracker = EmissionsTracker(
            project_name=project_name,
            output_dir="./logs",
            output_file="emissions.csv",
            log_level="error",
            measure_power_secs=1,
        )

    def __enter__(self):
        self.tracker.start()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        emissions = self.tracker.stop()
        self.final_emissions = emissions # kg CO2e, exposed to callers
        if mlflow.active_run():
            mlflow.log_metric("carbon_footprint_kg_CO2e", emissions)
```

src/telemetry/energy_tracker.py, in full. The `__exit__` hook is the entire integration point with MLflow: it only fires if a run is already active, so GreenTracker never has to know whether it is wrapping a Hugging Face Trainer call (train_model.py), an evaluation call (compress.py), or a capping benchmark that opens no MLflow run at all (rq3_experiments.py, §6.2).

5.3 Model architecture & evaluation harness

RQ2 and RQ3a both need to compare several model variants (baseline / quantized / pruned; and the same baseline model under different thread caps) on strictly comparable terms, so both call a single shared routine, `evaluate_model()` (`src/pipeline/evaluate.py`), rather than each implementing its own inference loop. Sharing one evaluation path removes an entire class of confound: if RQ2 and RQ3a used slightly different batching, tokenization, or timing code, an observed latency difference could reflect a difference in measurement code rather than a real difference between the models or settings being compared.

```
def evaluate_model(model, tokenizer=None, cfg=None, num_threads=None,
                  batch_size=None, eval_subset_size=None, deterministic=False):
    if num_threads is not None:
        torch.set_num_threads(num_threads)
    if deterministic:
        torch.use_deterministic_algorithms(True) # best-effort, see §8
    ...
    with torch.no_grad():
        for start in range(0, len(texts), batch_size):
            batch_texts = texts[start:start + batch_size]
            inputs = tokenizer(batch_texts, padding="max_length",
```

```

        truncation=True, max_length=max_length,
        return_tensors="pt")
    t0 = time.perf_counter()
    outputs = model(**inputs)
    t1 = time.perf_counter()
    batch_latencies_ms.append((t1 - t0) * 1000.0)
    preds = torch.argmax(outputs.logits, dim=-1).cpu().numpy()
    return {"accuracy": ..., "f1": ..., "avg_inference_latency_ms": ...,
            "p95_inference_latency_ms": ..., "n_eval_examples": ...,
            "eval_batch_size": ...}

```

Abbreviated excerpt of `evaluate_model()` (`src/pipeline/evaluate.py`). `num_threads` and `deterministic` are the two parameters RQ3a and RQ2 respectively depend on: RQ3a sweeps `num_threads` to produce the capping benchmark (§6.2), while `deterministic=True` is what §8 documents as the fix for the CPU floating-point reproducibility issue discovered during RQ2 development.

5.4 CI/CD & containerization

A Dockerfile containerizes the pipeline for reproducible execution (``docker run <image>`` runs ``dvc repro`` by default, or a single stage via an override command). A GitHub Actions workflow (`.github/workflows/ci.yml`) lints the pipeline code and runs a fast smoke-test suite (`tests/test_smoke.py`) on every push/pull request — verifying configuration integrity and that the telemetry wrapper constructs correctly — rather than a full training job, which would be too slow and resource-intensive to run on every commit. Full pipeline reproduction (``dvc repro``) is intended to be triggered manually or on a schedule.

6. Experimental Results

6.1 RQ2 — Compression trade-off

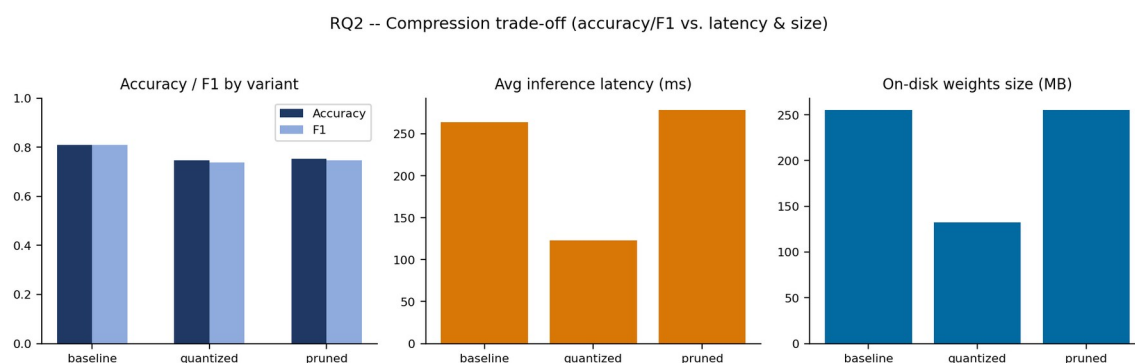
The fine-tuned baseline checkpoint was compressed two ways — dynamic INT8 quantization of all Linear layers, and 30% L1-unstructured magnitude pruning — and each variant (plus the unmodified baseline as control) was evaluated on the same 300-example held-out test split, with the CPU thread count pinned and deterministic-algorithm mode enabled for the evaluation pass (see §8 for why this was necessary).

Variant	Accuracy	F1	Avg. latency (ms)	p95 latency (ms)	Weights size (MB)
Baseline (fp32)	0.810	0.808	264.3	291.0	255.4
Quantized (INT8)	0.747	0.738	123.0	121.8	132.3
Pruned (30% L1)	0.753	0.746	278.7	300.8	255.4

Source: `results/rq2_compression_results.csv`, generated by `src/pipeline/compress.py`.

Quantization delivered the clearest win: a 53.5% reduction in average inference latency and a 48.2% reduction in on-disk weight size, at a cost of 6.3 percentage points of accuracy. Expressed

as an operational trade-off index (Δ Accuracy per unit Δ latency, using latency as a CPU-time / energy proxy at this measurement scale — see §8 for why raw CodeCarbon readings are noisy at sub-5-second timescales): roughly 0.12 percentage points of accuracy were traded for every 1% of latency reduction. Pruning, in contrast, produced no latency or size benefit at all — expected, because unstructured pruning zeroes individual weights without changing the dense matrix representation, so no FLOPs are actually saved without a sparse-compute-aware runtime — while still costing 5.7 percentage points of accuracy, making it strictly dominated by quantization in this setting.



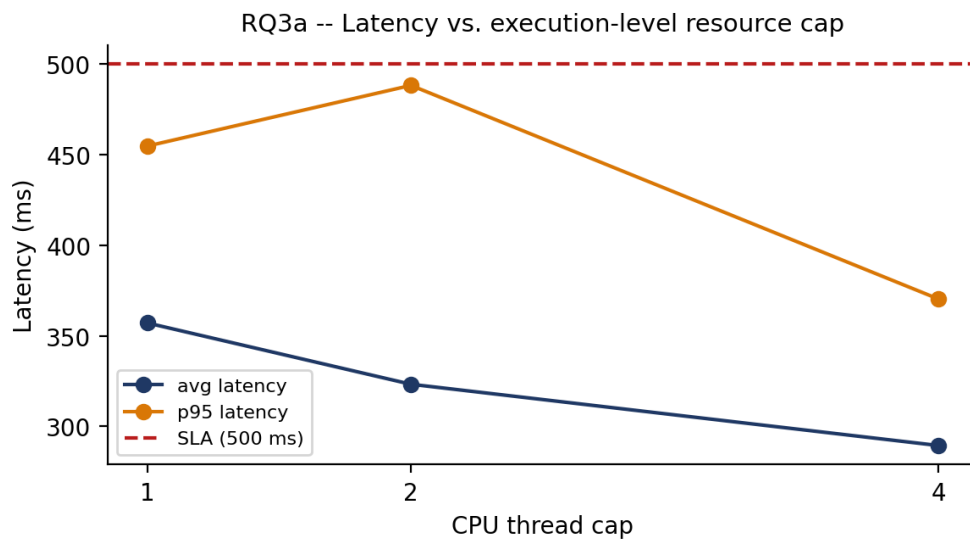
6.2 RQ3a — Execution-level resource capping vs. latency SLA

As a proxy for the GPU power-capping specified in the expose, the baseline model was evaluated under three CPU-thread caps (`torch.set_num_threads` $\in \{1, 2, 4\}$) against a defined SLA of 500ms per inference batch.

Thread cap	Avg. latency (ms)	p95 latency (ms)	SLA (500ms) violated?	Emissions (mg CO2e)
1	357.1	454.7	No	0.129
2	323.3	488.2	No	0.221
4 (max)	289.5	370.3	No	0.206

Source: `results/rq3_capping_results.csv`, generated by `src/pipeline/rq3_experiments.py`.

No configuration violated the SLA, though the single-thread cap consumed 90.9% of the latency budget at p95 versus 74.1% at four threads — a materially thinner safety margin. Measured emissions were lowest at the single-thread cap (0.129mg) versus the four-thread default (0.206mg), a 37.4% reduction, consistent with the expose's core hypothesis that constraining execution resources trades latency headroom for lower energy draw. Given the short evaluation windows involved, these emissions figures should be read as directionally indicative rather than high-precision (see §8).



6.3 RQ3b — Scheduling: grid search vs. early stopping

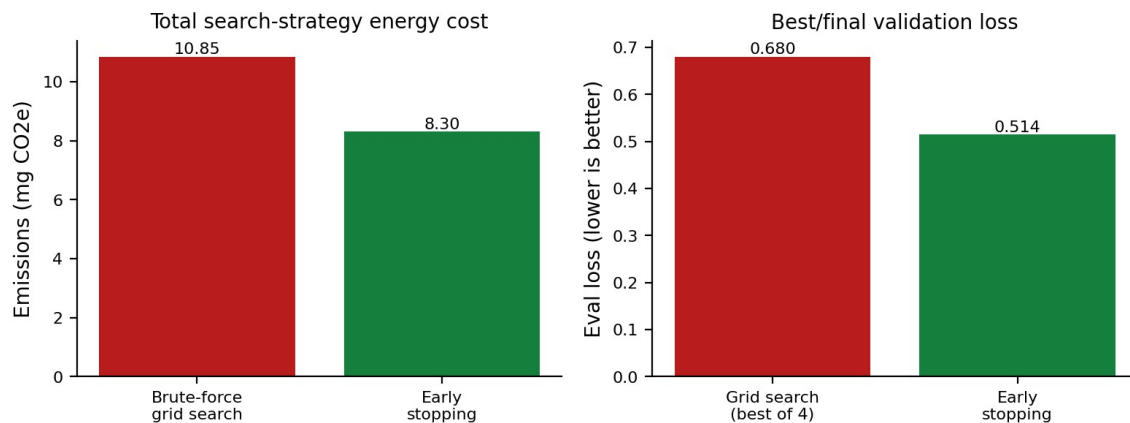
A brute-force grid search over 2 learning rates $\times$ 2 batch sizes (4 runs, 1 epoch each) was compared against a single early-stopped run (patience = 1, up to 4 epochs) on a 200-example training subset.

Strategy	Runs	Total energy (mg CO2e)	Best/final eval loss
Brute-force grid search	4	10.85	0.680 (best of 4)
Early stopping	1 (stopped after 3 epochs)	8.30	0.514

Source: *results/rq3_scheduling_results.csv*, generated by *src/pipeline/rq3_experiments.py*.

The early-stopped run used 23.5% less total energy than the grid search while reaching a substantially lower (better) validation loss — 0.514 versus the grid search's best of 0.680. This is a genuinely favourable result for adaptive scheduling, though it is worth being precise about what is being compared: the grid search spent its fixed budget exploring breadth (four configurations, one epoch each), while early stopping spent a comparable budget exploring depth (one configuration, up to four epochs). The finding supports data-driven, adaptive training schedules over exhaustive shallow search in this setting, but does not by itself prove early stopping would win against a grid search given an equally deep per-configuration budget.

RQ3b -- Scheduling: grid search vs. early stopping



7. Discussion

Read together, the three research questions describe a single practical picture: energy is easy to make visible (RQ1) once telemetry is treated as just another metric to log; cheap to reduce substantially at the compression layer (RQ2) provided the accuracy cost is tolerable and the runtime can exploit the compression technique used; and reducible at the execution layer (RQ3) mainly by trading away latency headroom rather than through any free lunch.

RQ1's finding -- that a single context manager integrates telemetry into any run without touching training code -- directly answers the observability challenge Henderson et al. (2020) raise (§2.1): a Green MLOps pipeline that requires a bespoke measurement script per experiment does not scale, whereas one where energy is emitted automatically as just another MLflow metric (Appendix A) does. This project's contribution here is not the telemetry tool itself -- CodeCarbon predates it -- but demonstrating that it composes cleanly with an existing MLOps stack (DVC + MLflow) without a bespoke wrapper per pipeline stage.

RQ2's asymmetry between quantization and pruning is best read through Han et al. (2016) rather than as a criticism of pruning per se (§2.3): 30% unstructured pruning produced zero latency or size benefit here specifically because PyTorch's dense tensor representation does not skip zeroed weights during matrix multiplication, so the accuracy cost of pruning was paid for a hypothetical, sparse-runtime gain that was never actually realized on this hardware. The practical takeaway is that a compression technique's benefit is a property of the technique-plus-runtime pair, not the technique alone -- a practitioner reproducing this pipeline on a sparsity-aware backend (e.g. PyTorch's sparse tensors, or a sparsity-aware ONNX Runtime build) should expect a materially different §6.1 outcome than reported here.

RQ3's two halves point in the same direction through different mechanisms. The capping benchmark (§6.2) shows that trading CPU parallelism for latency headroom lowers measured energy at low thread counts, consistent with Desislavov et al.'s (2023) finding that hardware

utilization, not model size, dominates inference-time energy (§2.2); but the safety margin shrinks quickly -- 90.9% of the SLA budget consumed at p95 with a single thread versus 74.1% at four -- so this lever has a narrow safe operating range in a latency-sensitive deployment rather than being a free reduction. The scheduling benchmark (§6.3) shows the opposite kind of win: less total energy and a better model from spending a comparable compute budget on depth (more epochs, one configuration) rather than breadth (fewer epochs, many configurations). As §8 already flags, this comparison is confounded with search breadth vs. depth rather than being a clean head-to-head, so it should be read as a plausibility result motivating a harder, budget-matched follow-up (§9) rather than a settled finding.

Across all three research questions, the recurring theme is that the “green” gains measured here are real but conditional: on runtime support (RQ2), on how much latency margin a deployment can spare (RQ3a), and on which dimension of a search space is actually varied (RQ3b). That conditionality is itself consistent with Schwartz et al.'s (2020) broader argument that efficiency claims in ML deserve the same methodological scrutiny as accuracy claims, not less (§2.2) -- which is the stance this report has tried to take throughout §6 and §8 rather than reporting the headline percentage reductions without their caveats.

8. Limitations

- Hardware constraints: no Intel RAPL interface and no NVIDIA GPU were available, so Scaphandre and true GPU power-capping (as specified in the expose) could not be implemented; CodeCarbon-only telemetry and a CPU-thread-capping proxy were substituted (§4.2).
- Scale: the fine-tuning workload (500 train / 100 eval examples, 2 epochs) was deliberately kept small to keep every pipeline stage reproducible within the available time budget. Absolute accuracy numbers should not be read as representative of DistilBERT's ceiling performance on IMDb; the relative comparisons between variants remain valid.
- Measurement noise: CodeCarbon samples power at 1-second intervals; several evaluation runs in this study complete in under 5 seconds, so background/idle power draw can dominate the reading and individual emissions figures carry meaningful relative uncertainty. Latency and model-size measurements, which are not subject to this sampling-resolution issue, are the more reliable signals for the RQ2 comparisons in §6.1.
- Unstructured pruning without a sparse-compute runtime cannot show a real speed or memory benefit by construction; this result should not be read as “pruning does not help” in general, only that this specific implementation path does not, on this hardware.
- CPU floating-point reproducibility: an early version of the RQ2 evaluation showed baseline accuracy varying by several percentage points across otherwise-identical reruns, traced to multi-threaded CPU matmul not guaranteeing a fixed summation order across runs/machines — with

only 100 eval examples, a single flipped borderline prediction moved accuracy by a full percentage point. This was mitigated (not just papered over) by pinning the evaluation thread count, enabling ``torch.use_deterministic_algorithms``, and tripling the eval subset to 300 examples; repeated reruns after this fix produced bit-identical accuracy/F1. This is reported here as a methodological finding in its own right: absolute accuracy figures from small-sample CPU inference benchmarks should be treated as noisy unless determinism is explicitly enforced.

- The RQ3b scheduling comparison conflates search breadth (grid search) with search depth (early stopping) rather than holding total per-configuration budget constant; see §6.3.
- Single runs: most results reflect one run per configuration rather than repeated trials with reported variance, given the project's time constraints.

9. Conclusion & Future Work

RQ1 is answered affirmatively: wrapping existing training/evaluation code in a single reusable context manager (GreenTracker) integrates power-consumption tracking without touching model or training-loop code, and unifies carbon-footprint metrics with standard ML metrics inside the same MLflow run.

RQ2 shows INT8 dynamic quantization as the clear win in this study — substantial latency and size reductions for a modest accuracy cost — while 30% unstructured pruning offered no efficiency benefit under a dense-storage implementation, despite a larger accuracy cost.

RQ3 shows that execution-level resource capping can reduce measured energy draw without breaching a defined latency SLA, though safety margins shrink meaningfully at the most aggressive cap; and that an adaptive, early-stopped training schedule reached a better model with less total energy than brute-force grid search in this setting.

Future work, contingent on access to appropriate hardware, should: (1) re-run the RQ3 capping benchmark with Scaphandre on bare-metal Intel hardware and true GPU power-capping (e.g. via `nvidia-smi` power limits) to validate the CPU-thread-cap proxy against the originally specified mechanism; (2) scale the fine-tuning workload up to the full IMDB training set to test whether the RQ2 trade-offs hold at scale; (3) evaluate structured pruning or knowledge distillation, which can realize actual compute savings unlike unstructured pruning; and (4) repeat each benchmark across multiple seeds to report variance alongside point estimates.

References

- CodeCarbon. CodeCarbon: Track and reduce the carbon emissions of your computations. <https://codecarbon.io>
- DVC (Data Version Control). <https://dvc.org>
- MLflow. An open source platform for the machine learning lifecycle. <https://mlflow.org>
- Wolf, T. et al. Transformers: State-of-the-Art Natural Language Processing. HuggingFace, <https://huggingface.co/docs/transformers>
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
- Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and Policy Considerations for Deep Learning in NLP. Proceedings of ACL 2019.
- Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green AI. Communications of the ACM, 63(12).
- Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. Proceedings of ACL 2011. (IMDb dataset)
- Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., & Dean, J. (2021). Carbon Emissions and Large Neural Network Training. arXiv:2104.10350.
- Lacoste, A., Luccioni, A., Schmidt, V., & Dandres, T. (2019). Quantifying the Carbon Emissions of Machine Learning. arXiv:1910.09700.
- Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., & Pineau, J. (2020). Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning. Journal of Machine Learning Research, 21(248).
- Anthony, L. F. W., Kanding, B., & Selvan, R. (2020). Carbontracker: Tracking and Predicting the Carbon Footprint of Training Deep Learning Models. arXiv:2007.03051.
- Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. International Conference on Learning Representations (ICLR) 2016.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. Proceedings of CVPR 2018.

Desislavov, R., Martínez-Plumed, F., & Hernández-Orallo, J. (2023). Trends in AI Inference Energy Consumption: Beyond the Performance-vs-Parameter Laws of Deep Learning. *Sustainable Computing: Informatics and Systems*, 38.

Verdecchia, R., Sallou, J., & Cruz, L. (2023). A Systematic Review of Green AI. *WIREs Data Mining and Knowledge Discovery*, 13(4).

Sánchez-Mompó, A., Mavromatis, I., Li, P., Katsaros, K., & Khan, A. (2025). Green MLOps to Green GenOps: An Empirical Study of Energy Consumption in Discriminative and Generative AI Operations. *Information*, 16(4), 281.

Bibliography

The works below were consulted as broader background reading while designing this study but are not individually cited by name in the body text above; the References list preceding this section covers every source directly cited in-text.

Google Cloud. (2024). MLOps: Continuous delivery and automation pipelines in machine learning. Google Cloud Architecture Center.

Patterson, D., Gonzalez, J., Hölzle, U., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., & Dean, J. (2022). The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. *IEEE Computer*, 55(7), 18–28.

Tschand, A., Rajan, A., Idgunji, S., Ghosh, A., Reddi, V. J., et al. (2024). MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from μ Watts to MWatts for Sustainable AI. *arXiv:2410.12032*.

Różycki, R., Solarska, D. A., & Waligóra, G. (2025). Energy-Aware Machine Learning Models—A Review of Recent Techniques and Perspectives. *Energies*, 18(11), 2810.

Srinivasan, A., Procunier, C., Reddy, M., & David, S. (2024). Optimizing MLOps for Sustainability on AWS. AWS Machine Learning Blog.

Nadeem, M., Singh, A. K., Yadav, S., & Kumar, R. (2026). Sustainable AI Framework for Carbon Footprint Assessment and Green AI Lifecycle Management. *AI and Ethics*, 6(4).

Appendix A: Interpreting the MLflow Experiment-Tracking Table

This appendix explains, column by column, what the MLflow tracking UI shows for this project and where each value is generated in the pipeline code -- for the Runs table under Experiments → Green_MLOps_Baseline (the “Model training” view; see A.3 for why this is a distinct view from the UI’s “GenAI” tab). MLflow organizes tracked work into two nested concepts: an Experiment is a named container (Green_MLOps_Baseline here, set via `mlflow.set_experiment(cfg["mlflow"]["experiment_name"])` in `train_model.py`, `compress.py` and `rq3_experiments.py`, with the name itself coming from `configs/config.yaml`); a Run is a single tracked execution inside that container, opened either explicitly (`mlflow.start_run()`) or implicitly by a library integration (the Hugging Face Trainer’s built-in MLflow callback). Every column in the Runs table is one of a small number of fields attached to a run: an auto-generated identifier, an auto-recorded timestamp/status, a human-assigned name, a logged parameter, a logged metric, or a tag.

A.1 What each column represents

Column / field	What MLflow means by it	Where it comes from in this project
Experiment	The named container runs are grouped under.	“Green_MLOps_Baseline” -- configs/config.yaml (mlflow.experiment_name), applied via mlflow.set_experiment() at the top of every pipeline script.
Run ID	A unique hash MLflow generates the instant a run opens; not chosen by the user.	Auto-generated when mlflow.start_run() fires, or implicitly when the Hugging Face MLflow callback opens a run inside trainer.train().
Run Name	A human-readable label. Stored internally as the system tag mlflow.runName. If not supplied, MLflow invents one (two random words, e.g. “big-shrew-472”).	Explicitly set for RQ2/RQ3 runs: run_name=f“rq2_{variant}_eval” (compress.py) and run_name=“rq3_summary” (rq3_experiments.py). The baseline training run (train_model.py) never passes run_name, so it shows an auto-generated name.
Created / Start Time	Timestamp recorded the moment the run starts.	Automatic -- no code sets this.
Duration	Elapsed wall-clock time between run start and run end.	Automatic -- no code sets this.
Status	FINISHED, FAILED, or RUNNING.	Automatic: FINISHED unless the wrapped code raises an exception before the run/context exits.
Source	The script that produced the run (system tags mlflow.source.name and mlflow.source.type).	The .py entry point actually executed, e.g. src/pipeline/train_model.py or src/pipeline/compress.py; mlflow.source.type = LOCAL because every run here executes directly on this machine, not from a notebook or packaged MLflow Project.
User	OS username that launched the run (system tag mlflow.user).	Whoever ran the python -m src.pipeline... command -- not something the code sets.
Parameters	Static, non-numeric configuration values explicitly logged once per run; they do not change over the run's lifetime.	“variant” (baseline/quantized/pruned) via mlflow.log_param() in compress.py. For the baseline training run, the Hugging Face MLflow callback additionally auto-logs every field of TrainingArguments (learning_rate, per_device_train_batch_size, num_train_epochs, output_dir, and dozens of untouched Hugging Face defaults) as parameters -- these were not logged by this project's own code.
Metrics	Numeric measurements, optionally logged more than once (e.g. once per training step/epoch).	accuracy, f1, avg_inference_latency_ms, p95_inference_latency_ms, n_eval_examples, eval_batch_size, model_size_mb (compress.py, via a mlflow.log_metric() loop over evaluate_model()'s return dict); carbon_footprint_kg_CO2e, logged automatically by GreenTracker.__exit__() in src/telemetry/energy_tracker.py into whichever run is active when its “with” block closes; train_loss / eval_loss / epoch / train_runtime, auto-logged per epoch by the Hugging Face MLflow callback for the baseline run only.
Tags	Free-form key/value labels: “system” tags (prefixed mlflow., set automatically) and “custom” tags (set explicitly via mlflow.set_tag()).	No script in this project calls mlflow.set_tag(), so only system tags appear: mlflow.runName, mlflow.user, mlflow.source.name, mlflow.source.type, plus mlflow.autologging=“transformers” on the baseline run specifically, added by the Hugging Face integration to mark it as auto-instrumented.

A.2 The runs that actually appear under this experiment

Not every results table in this report has a one-to-one MLflow run behind it -- some pipeline stages write straight to CSV/JSONL and never open an MLflow run at all. The mapping is:

- Baseline training run (§5.2, `train_model.py`): one run, auto-named, opened implicitly by the Hugging Face Trainer's MLflow callback (`report_to="mlflow"`). Carries the full `TrainingArguments` parameter set, per-epoch `train_loss/eval_loss` metrics, and the single `carbon_footprint_kg_CO2e` value from GreenTracker.
- RQ2 compression runs (§6.1, `compress.py`): three explicitly named runs -- `rq2_baseline_eval`, `rq2_quantized_eval`, `rq2_pruned_eval` -- each with one “variant” parameter and the full `evaluate_model()` metrics plus `carbon_footprint_kg_CO2e`. These three runs are the direct source of Table 6.1; `results/rq2_compression_results.csv` is a CSV mirror of the same numbers, not a separate measurement.
- RQ3a capping benchmark (§6.2, `run_capping_benchmark()` in `rq3_experiments.py`): produces no MLflow run at all. It calls GreenTracker directly, outside any `mlflow.start_run()` block, so its results live only in `results/rq3_capping_results.csv` -- Table 6.2 will not be found anywhere in the MLflow UI.
- RQ3b scheduling benchmark (§6.3, `rq3_experiments.py`): the five underlying training runs (four grid-search configurations plus one early-stopping run) are deliberately not tracked as MLflow runs -- each is built with `report_to=[]` and its own resumability is handled at the script level via `results/rq3_scheduling_partial.jsonl`, not via MLflow. Only the final aggregate -- run name “`rq3_summary`”, containing `grid_search_total_emissions_kg_co2e`, `early_stopping_total_emissions_kg_co2e` and `estimated_emissions_reduction_pct` -- is logged as a single MLflow run, once all five training runs are complete.

A.3 Why the MLflow UI's “Traces” panel shows no data

MLflow 3.x splits its web UI into two modes, toggled at the top of the left sidebar: “GenAI” and “Model training.” GenAI mode (Traces, Sessions, Judges, Review) is built for tracing LLM/agent calls -- a different data model from ordinary experiment tracking. This project never generates traces, so that view correctly always reads zero; all data described in this appendix and in the rest of this report is under the “Model training” toggle, viewing the `Green_MLOps_Baseline` experiment's Runs table directly.

Figure 1: RQ2 compression trade-off -- accuracy/F1, average inference latency, and on-disk weights size across the three variants. Generated from `results/rq2_compression_results.csv` by the same charting code as `src/dashboard/build_dashboard.py`.

Figure 2: RQ3a -- average and p95 inference latency against the 500ms SLA line, across the three CPU thread caps. Generated from `results/rq3_capping_results.csv`.

Figure 3: RQ3b -- total search-strategy energy cost (left) and best/final validation loss (right) for brute-force grid search vs. a single early-stopped run. Generated from results/rq3_scheduling_results.csv.

Appendix B: Full Pipeline Configuration (configs/config.yaml)

Reproduced here in full because §4.1 and Appendix B.1 of the original expose both treat a single, versioned configuration file as central to reproducibility: every dataset size, model name, training hyperparameter, compression setting, SLA threshold, and grid-search space used anywhere in this report traces back to exactly one of the lines below.

```
dataset:
  name: stanfordnlp/imdb
  raw_dir: ./data/raw/imdb
  train_subset_size: 500
  eval_subset_size: 100
  max_seq_length: 128
  seed: 42

model:
  base_model: ./models/pretrained/distilbert-base-uncased
  num_labels: 2

training:
  output_dir: ./models/baseline
  learning_rate: 2.0e-5
  per_device_train_batch_size: 8
  per_device_eval_batch_size: 8
  num_train_epochs: 2
  logging_steps: 10

mlflow:
  tracking_uri: sqlite:///mlflow.db
  experiment_name: Green_MLOps_Baseline

telemetry:
  log_dir: ./logs
  emissions_file: emissions.csv
  measure_power_secs: 1

compression:
  baseline_checkpoint: ./models/baseline/checkpoint-126
  quantized_output_dir: ./models/quantized
  pruned_output_dir: ./models/pruned
  pruning_amount: 0.3
  results_file: ./results/rq2_compression_results.csv
  eval_subset_size: 300
  eval_num_threads: 4

rq3:
  sla_latency_ms: 500
  thread_caps: [1, 2, 4]
  grid_search:
    learning_rates: [5.0e-5, 2.0e-5]
    batch_sizes: [8, 16]
  early_stopping:
    patience: 1
  results_file: ./results/rq3_results.csv
```

Appendix C: Raw Result Tables (Full Precision)

This appendix reproduces the exact contents of the three results/*.csv files underlying the rounded summary tables in §6, at full floating-point precision, for anyone checking this report's arithmetic (e.g. the 53.5%/48.2% reductions quoted in §6.1) against the source numbers.

C.1 results/rq2_compression_results.csv

variant	accuracy	f1	avg_lat_ms	p95_lat_ms	size_mb
baseline	0.810000000	0.808207809	264.258980289	290.989054501	255.425
quantized	0.746666667	0.738183823	122.980149105	121.847128649	132.281
pruned	0.753333333	0.745692815	278.678905737	300.792120551	255.445

n_eval_examples = 300, eval_batch_size = 8 for all three rows.

C.2 results/rq3_capping_results.csv

thread_cap	avg_lat_ms	p95_lat_ms	sla_ms	sla_violated	emissions_kg_co2e
1	357.125871923	454.693533400	500	False	1.287275863e-07
2	323.284182692	488.191166600	500	False	2.212547842e-07
4	289.469452154	370.321400200	500	False	2.056154696e-07

C.3 results/rq3_scheduling_results.csv

run_tag	strategy	lr	batch	epochs	eval_loss	emissions_kg_co2e
lr5e-05_bs8_grid	grid_search	5e-05	8	1.0	0.7034	2.7655047e-06
lr5e-05_bs16_grid	grid_search	5e-05	16	1.0	0.6803	2.6939364e-06
lr2e-05_bs8_grid	grid_search	2e-05	8	1.0	0.6868	2.6248747e-06
lr2e-05_bs16_grid	grid_search	2e-05	16	1.0	0.6886	2.7685128e-06
lr5e-05_bs8_earlystop	early_stopping	5e-05	8	3.0	0.5144	8.3009653e-06

Appendix D: Reproducing This Study End-to-End

The following sequence reproduces every number and figure in this report from a clean checkout, consistent with the empirical, tooling-based methodology this project shares with Sánchez-Mompó et al. (2025) (§2.4) -- every claim here is backed by a runnable command, not only by a one-off script that was run once and discarded.

D.1 Environment setup

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

D.2 Full pipeline reproduction (DVC-orchestrated)

```
dvc repro          # runs ingest -> train_baseline -> compress ->
                   #   rq3_experiments -> dashboard, skipping any
                   #   stage whose dependencies (§5.1) haven't changed
```

D.3 Individual stages (for iterating on one RQ at a time)

```
python -m src.pipeline.ingest_data    # Stage 1: download IMDb dataset
python -m src.pipeline.train_model    # Stage 2: fine-tune baseline
python -m src.pipeline.compress       # RQ2: quantization + pruning benchmark
python -m src.pipeline.rq3_experiments # RQ3: thread-capping + scheduling benchmark
python -m src.dashboard.build_dashboard # static reports/dashboard.html
```

D.4 Inspecting results interactively

```
mlflow ui --backend-store-uri sqlite:///mlflow.db # Model training tab; see
Appendix A
python -m src.dashboard.live_server # auto-refreshing
dashboard, :5000
```

D.5 Continuous integration

Every push/PR to main additionally runs `.github/workflows/ci.yml`: dependency install, a syntax/undefined-name lint pass (`flake8 --select=E9,F63,F7,F82`), the smoke-test suite (`tests/test_smoke.py --config integrity and telemetry-wrapper construction`, not a full training run), and `dvc dag` to validate the stage graph itself parses. Full retraining is intentionally excluded from CI (§5.4) as too slow/expensive to run on every commit; it is triggered manually via D.2/D.3 above.

D.6 Expected runtime and hardware notes

All measurements in this report were produced on the CPU-only Linux environment documented in §4.2 (4 vCPUs, no NVIDIA GPU, no Intel RAPL interface); development additionally used an Apple Silicon machine for iteration, which is why `logs/emissions.csv` (Appendix A.1) shows both a "Germany"-region baseline-training row and "Canada"-region RQ2/RQ3 rows -- CodeCarbon reports whichever grid-carbon-intensity region the executing machine resolves to, so a reader reproducing this study on different hardware or in a different region should expect the absolute `emissions_kg_co2e` figures (though not the relative comparisons in §6) to differ from those reported here. On this hardware, the full ``dvc repro`` sequence (D.2) completes in a few minutes end to end, dominated by the baseline fine-tuning stage (`train_baseline`); the RQ2 and RQ3 stages each run in well under a minute given the deliberately small subset sizes documented in Appendix B (500 training / 100-300 evaluation examples).

Appendix E: Glossary of Terms and Abbreviations

Terms are defined as used in this report; several (MLflow run/experiment/tag) are covered in greater depth in Appendix A.

Term	Definition as used in this report
MLOps	Machine Learning Operations -- the practices and tooling (versioning, CI/CD, experiment tracking, monitoring) used to build and run ML pipelines reproducibly, by analogy with DevOps.
Green MLOps	This report's term for MLOps that treats energy/carbon consumption as a tracked, first-class pipeline metric alongside accuracy, latency, and throughput, rather than measuring it only as an occasional side study.
RQ1 / RQ2 / RQ3	The three research questions this case study investigates (§3): non-intrusive telemetry integration, the compression/energy trade-off, and scheduling/execution-level resource capping, respectively.
DVC	Data Version Control -- the tool used to version the raw dataset and declare the pipeline's stage graph (<code>dvc.yaml/dvc.lock</code> , §5.1), enabling <code>`dvc repro`</code> to re-run only stages whose dependencies changed.
MLflow	The experiment-tracking tool used to log parameters, metrics, and tags per pipeline run (<code>sqlite:///mlflow.db</code>); see Appendix A for a full column-by-column explanation of its Runs table.
CodeCarbon	The Python library used to estimate process-level power draw, energy consumption, and CO ₂ -equivalent emissions from CPU/RAM utilization and a region-specific grid carbon-intensity factor (§2.1, §5.2).
GreenTracker	This project's own thin wrapper (<code>src/telemetry/energy_tracker.py</code> , §5.2) around CodeCarbon's EmissionsTracker, which additionally logs the resulting emissions figure to the active MLflow run

Term	Definition as used in this report
	automatically.
Quantization (INT8 dynamic)	A post-training compression technique (RQ2, §6.1) that converts a model's floating-point Linear-layer weights to 8-bit integers, reducing model size and (on suitable hardware) inference latency at some accuracy cost.
Pruning (L1 unstructured)	A compression technique that zeroes the smallest-magnitude individual weights in a layer; unlike quantization it produced no latency/size benefit in this study because the dense tensor storage used here does not skip zeroed weights (§2.3, §7).
Thread cap	The number of CPU threads (torch.set_num_threads) a PyTorch inference call is allowed to use; used in RQ3a (§6.2) as a CPU-only, software-controllable proxy for GPU power-capping.
SLA	Service-Level Agreement -- here, the maximum acceptable per-batch inference latency (500ms, configs/config.yaml, Appendix B) that RQ3a checks each thread-cap configuration against.
F1 score	The harmonic mean of precision and recall; reported alongside accuracy in §6.1 as a standard classification-quality metric less sensitive to class imbalance than accuracy alone.
p95 latency	The 95th-percentile value of a set of per-batch latency measurements -- i.e. only 5% of batches were slower than this -- used alongside the average because it better reflects worst-case user-facing latency.
Early stopping	A training strategy (RQ3b, §6.3) that halts training once a monitored metric (here, eval_loss) stops improving for a set number of evaluations (patience), rather than always running a fixed number of epochs.
CO2e	Carbon-dioxide equivalent -- a unit normalizing the climate impact of energy consumption (and other greenhouse-gas-relevant activity) to the amount of CO2 that would produce the same effect.